

Fast End-to-End Simulation and Exploration of Many-RISCV-Core Baseband Transceivers for Software-Defined Radio-Access Networks

Marco Bertuletti
ETH Zürich
Zürich, Switzerland
mbertuletti@iis.ee.ethz.ch

Yichao Zhang
ETH Zürich
Zürich, Switzerland
yiczhang@iis.ee.ethz.ch

Mahdi Abdollahpour
Università di Bologna
Bologna, Italy
mahdi.abdollahpour@unibo.it

Samuel Riedel
ETH Zürich
Zürich, Switzerland
sriedel@iis.ee.ethz.ch

Alessandro Vanelli-Coralli
ETH Zürich
Zürich, Switzerland
Università di Bologna
Bologna, Italy
avanelli@iis.ee.ethz.ch

Luca Benini
ETH Zürich
Zürich, Switzerland
Università di Bologna
Bologna, Italy
lbenini@iis.ee.ethz.ch

Abstract—The fast-rising demand for wireless bandwidth [1] requires rapid evolution of high-performance baseband processing infrastructure. Programmable many-core processors for software-defined radio (SDR) have emerged as high-performance baseband processing engines, offering the flexibility required to capture evolving wireless standards and technologies [2]–[4]. This trend must be supported by a design framework enabling functional validation and end-to-end performance analysis of SDR hardware within realistic radio environment models. We propose a static binary translation based simulator augmented with a fast, approximate timing model of the hardware and coupled to wireless channel models to simulate the most performance-critical physical layer functions implemented in software on a many (1024) RISC-V cores cluster customized for SDR. Our framework simulates the detection of a 5G OFDM-symbol on a server-class processor in 9.5s-3min, on a single thread, depending on the input MIMO size (three orders of magnitude faster than RTL simulation). The simulation is easily parallelized to 128 threads with 73-121× speedup compared to a single thread.

Index Terms—RISC-V, SDR, 5G, 6G, SBT.

I. INTRODUCTION

The worldwide demand for more wireless subscriptions (8.5 billion in 2023) and network data traffic (145EB in Q1-2024) [1] fostered the development of new Radio-Access-Networks (RAN) technologies. The Massive Multiple Input Multiple Output (MIMO) paradigm ratified by 3rd Generation Partnership Project (3GPP) Release-17 [5] gained momentum and will extend to more users and antennas in the upcoming 6th Generation (6G) releases [6]. Alongside increased performance requirements, channel traffic characteristics will also change, resulting in a dynamic range from a few kBs for a voice call to GBs for high-quality multimedia [7].

The flexibility required to address demanding worst-case specifications and wide operation ranges can be obtained

by implementing the MIMO processing workload on programmable cores, an approach known as Software Defined Radio (SDR). Multi-core and many-core systems promise high performance on SDR 5th Generation (5G) workloads at the edge, implementing the full Physical Layer (PHY) on basestations (BSs) [2]–[4]. Additionally, the RISC-V Instruction Set Architecture (ISA) emerged as a powerful domain-specialization tool to fight back the energy-efficiency loss of instruction processors vs. custom hardware, enabling the co-design of efficiency-boosting processors features and domain-specific instruction extensions [3], [4].

Evaluating the algorithmic (functional) performance of software running on SDR-hardware and its execution performance under varying network traffic, spatial topology, and transmission channel conditions is critical to the design of future BSs. To this purpose, a model of the SDR receivers must be simulated as Device Under Test (DUT) in an end-to-end (E2E) transmission, matching the following requirements: deterministic behavior, flexibility to evolving standards, and awareness of timing to synchronize with other elements of the processing chain [7]. Additionally, a fast simulation framework is needed to evaluate the receiver performance via extensive numerical Monte Carlo (MC) simulation.

TABLE I
SIMULATION METHODS FOR SDR BASEBAND HARDWARE.

	Device	Freq.	Speed	Design Effort	Multi-Core
[8], [9]	RTL	-	↓	↓	⊗
[10]	TLM	Intel Core2	3.00GHz	↑	⊗
[11]	FPGA	XCZU28DR	128MHz	↓	⊗
[2]	FPGA	ZCU102	120MHz	↑	⊗
Ours	SBT	AMD EPYC-7742	3.25GHz	↓	⊙

Table I summarizes different options to include SDR hard-

ware as DUT in an E2E wireless transmission testbench. Event-based (e.g., QuestaSim [12]) and compiled (e.g., Verilator [13]) Register Transfer Level (RTL) simulators, require low effort to directly simulate Hardware Description Language (HDL) code of baseband receivers [8], [9], [14]. Unfortunately, they are too slow for MC simulation of complex workloads. Transaction Level Modeling (TLM) in SystemC allows the creation of detailed virtual prototypes of entire digital systems [10] and can also be used for the simulation of RF analog transceivers [15]–[17]. However, TLM simulations are limited in speed for the sequential execution of SystemC processes managed by a centralized kernel [18] and require long compilation time, constraining design iterations. Field Programmable Gate Array (FPGA) [2], [11] or Application Specific Integrated Circuit (ASIC) prototyping can not be used to implement a fast hardware-software co-design loop, because it requires a fully designed system and can be used only late in the design process. In the case of complex baseband many-core receivers, the prototyping on FPGA also requires considerable design effort. High Level Synthesis (HLS) tools, such as Python Productivity for Zynq (PYNQ) [11], are used to emulate small dataflows on FPGA, but the approach does not extend to many-core baseband processors with hundreds of processing elements. Moreover, implementing multi-core systems on FPGA must trade off limited hardware resources and operating frequencies, as clearly shown by the implementation of 36 RI5CY cores on Xilinx XCVU9, presented in [2].

This paper proposes a lightweight Static Binary Translation (SBT) based framework to simulate SDR many-core programmable transceivers based on the RISC-V ISA. Our simulator can be connected to channel models for extensive MC analysis and is easily parallelized on Commercial Off The Shelf (COTS) multi-core servers. The following contributions quantify these claims:

- We propose the open-source¹ SBT-based simulator Ban-shee [19] for deterministic and instruction accurate emulation of many-core transceivers. The simulator is connected to wireless channel models [20] for E2E MC analysis of the receiver performance. As a challenging benchmark, we emulate the 1024 independent instruction streams of the open-source² programmable TeraPool-SDR [3] on a 128-cores AMD EPYC-7742 server.
- We demonstrate fast design space exploration on the Minimum Mean Squared Error (MMSE) detection in different arithmetic precisions. A single thread of the 128-core AMD EPYC-7742 server simulates the detection of an Orthogonal Frequency Division Multiplexing (OFDM) symbol in < 3min, depending on the MIMO size. A small 4x4 input only requires 9.5s, compared to 13h:44min for RTL simulation of the same binary (Questasim-2022.3). Independent symbols are easily parallelized, with 73-121× speedup, compared to a single thread.

- We show that augmenting the SBT-based simulator with an approximate timing model of the hardware offers first-order estimates of the receiver processing time, with 30% average error over the measured RTL cycle-count for different MMSE implementations and MIMO sizes.

Running MMSE detection with a speedup over RTL simulation exceeding 1000×, the developed framework enables fast MC analysis of programmable baseband transceivers.

II. MANY-CORE SDR ARCHITECTURE

TeraPool [3], the largest shared-memory many-core cluster for SDR presented in the open literature, is a challenging simulation benchmark: emulating its 1024 independent instruction streams, diverse ISA, and large hierarchical interconnect is time-consuming in RTL simulation and would require consistent modeling effort in TLM. Moreover, TeraPool’s open-source RTL design enables comparisons with cycle-accurate simulation.

The TeraPool architecture, illustrated in Figure 1, features *Snitch* processing elements that support the RV32IMAF ISA. The single-stage core decodes instructions and executes the RV32I ISA, excluding load&stores. Other instructions are offloaded to co-processing functional units, including a Load&Store Unit (LSU), an Integer Processing Unit (IPU) for integer multiplication and division, and a Floating Point Unit (FPU) implementing the *zfinx* and *zhinx* extensions [21].

Snitch’s functional units also implement custom integer and Floating Point (FP) RISC-V extensions for Digital Signal Processing (DSP). The *Xpulpimg* set includes post-increment load&stores, Multiply and Accumulates (MACs), and Single Instruction Multiple Data (SIMD) compute and data-shuffling operations on 8b-16b integer types. The *Smallfloat* and *Mini-float* sets [22], [23] include SIMD operations on FP 8b-16b types, widening-dotproduct (wDotp) with 8b-16b operands and 16b-32b accumulators, and complex 16b FP MAC.

The base of TeraPool’s hierarchical design is the *Tile*, where 8 cores share 32 KiB of scratchpad memory with 1-cycle access latency and 4 KiB of I\$. The cores access the scratchpad of other Tiles via shared ports to the cluster interconnects. One 8x8 crossbar connects 8 Tiles in the *SubGroup* hierarchy. Three 8x8 crossbars connect Tiles of the SubGroup to Tiles of 3 other SubGroups in the *Group* hierarchy. Three 32x32 crossbars connect the Tiles of a Group to Tiles in 3 other Groups, summing up to 128 Tiles and 4 MiB of scratchpad in a Cluster. Pipeline stages at hierarchy boundaries configure non-uniform memory access (less than 9 cycles without contentions). A cluster-level AXI interconnect and a Direct Memory Access (DMA) engine allow explicit data transfers from L2 memory.

III. CO-SIMULATION OF DUT WITH TX-CHANNEL

We describe a wireless E2E MIMO transmission and the developed framework for co-simulation of the transmission channel model and of the RISC-V DUT.

¹<https://github.com/pulp-platform/banshee>

²<https://github.com/pulp-platform/mempool>

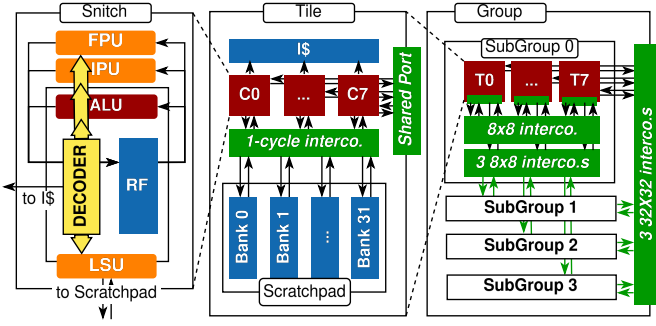


Fig. 1. Snitch core, Tile, SubGroup, and Group of TeraPool.

A. End-to-end MIMO wireless transmission

MIMO is a key enabling technology for 5G and beyond networks, where the BS is equipped with N_{RX} antenna elements, which receive signals from N_{TX} user equipments (UEs). During uplink, the transmission bit sequence maps to a Quadrature-Amplitude Modulation (QAM) constellation, and blocks of QAM-symbols are transmitted through the channel on N_{SC} OFDM subcarriers, namely an OFDM-symbol [5]. At the BS, the detection algorithm uses channel state information (CSI) to estimate the transmitted QAM-symbols. The BS processes a Transmission Time Interval (TTI) with 14 OFDM-symbols in $<1\text{ms}$. The evolution from 5G to 6G will increase in the N_{SC} , N_{TX} , and N_{RX} , affecting the Key Performance Indicators (KPIs) of DSP detection algorithms as well as their hardware implementation.

We therefore identify MIMO detection as a reference application to prove the capabilities of our co-simulation approach. We implement the MMSE detector, which offers optimal performance for transmission through a Additive White Gaussian Noise (AWGN) channel, on the target SDR-hardware.

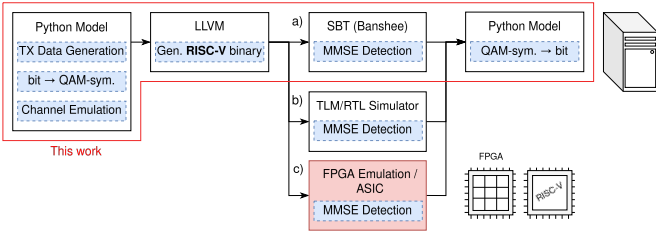


Fig. 2. Different approaches to add RISC-V hardware in the loop of a telecommunication application: c) execution on prototype ASIC, emulation on FPGA, b) cycle-accurate RTL/TLM simulation and a) simulation on Banshee.

B. Simulation of the MIMO wireless transmission

The proposed framework and flow are described in Figure 2. A Python model of the MIMO transmission based on [20] runs on the simulation host. The model includes bit-sequence transmission generation for different users, QAM-symbol mapping, and wireless channel emulation. It also implements a double-precision reference of the detection MMSE algorithm.

The SDR hardware running MMSE processing is simulated on Banshee, an open-source LLVM-based binary translator for instruction-accurate emulation of multi-core systems. The

simulation flow in Banshee is divided into two phases. During *translation*, Banshee converts a RISC-V binary into LLVM Intermediate Representation (IR) via SBT. The generated IR references to Banshee runtime functions to keep track of the emulated device state. LLVM optimizes and translates the IR into host code. During *emulation*, Banshee maps the translated code to multiple threads on the host platform for each hardware thread of the emulated RISC-V multi-core.

Besides offering an instruction-accurate simulation of the host and taking advantage of fully parallel execution, Banshee assigns a static latency to each instruction to estimate the program runtime. To model outstanding instruction execution, Banshee implements a scoreboard that keeps track of the Read After Write (RAW) dependencies between instructions. This feature allows a fast software performance estimation by accounting for instruction stalls caused by long-latency instructions such as loads&stores.

We modified Banshee to translate *zfinx*, *zhix*, *Smallfloat*, and *Minifloat* FP extensions, which compute on operands in the integer register file, as specified in [21]. The latency of FP instructions is also annotated in the simulator for first-order estimation on the emulated code runtime.

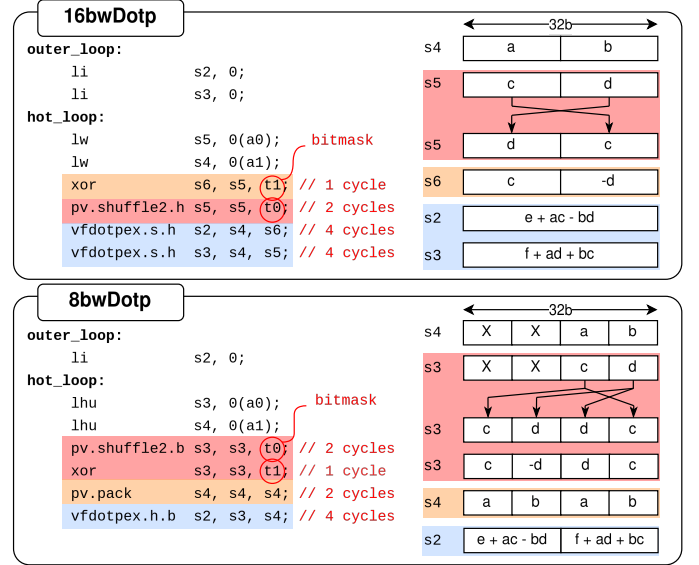


Fig. 3. Execution of the complex MAC in different arithmetic precisions using *Smallfloat* and *Xpulpig* instruction sets.

IV. SOFTWARE-DEFINED MMSE ON TERAPOL

This section describes the SDR implementation of a MMSE detector on TeraPool. Given y the received signal, $\hat{H}$ and σ an estimation of the channel and noise at the BS, and $\hat{x}$ the detected signal, the analytical MMSE detector is:

$$\hat{x}_i = (\hat{H}_i^H \hat{H}_i + \sigma_i^2 I)^{-1} \hat{H}_i^H y_i \quad \forall i \in [1, N_{SC}] \quad (1)$$

The operator in Equation (1) requires a matrix inversion. Given $G = (\hat{H}^H \hat{H} + \sigma^2 I)$ and $z = \hat{H}^H y$, we decompose $G = L^H L$ in its lower and upper triangular components, using the Cholesky algorithm. The linear system $G\hat{x} = z$ is solved by

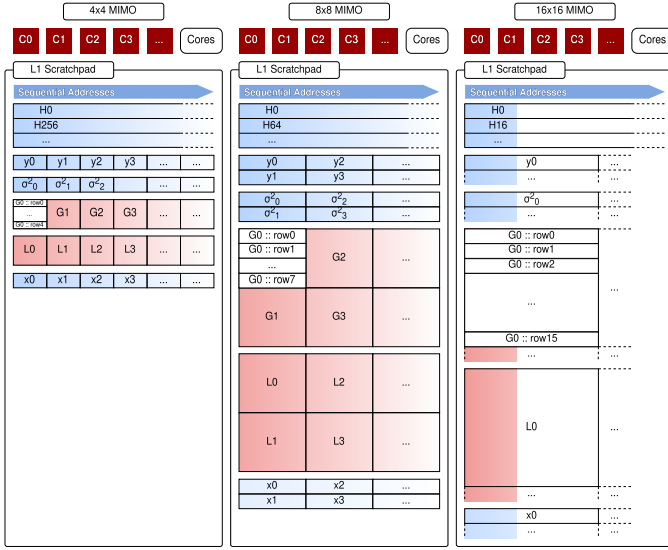


Fig. 4. Implementation of parallel MIMO-MMSE on the TeraPool many-core cluster for different sizes of the MIMO problem.

inverting two lower triangular matrices $\hat{x} = L^{-1}((L^H)^{-1}z)$. This reduces the overall arithmetic complexity of the inversion, as described in [24]. The MMSE detection then requires four operators: the Hermitian of the channel, the Matrix Vector Multiplication (MVM) between the complex-conjugate of the channel and the received symbol, the Cholesky decomposition, and the solution of a triangular system.

TeraPool has a *fork-join* type programming model (common trait of many-core clusters). In the *fork* phase of the program, each core executes an independent instruction stream in parallel. The *join* phase consists of a synchronization barrier: cores wait for each other before executing the next parallel section.

In an OFDM transmission a different MMSE problem must be solved for each subcarrier of the transmission spectrum. In the parallel execution on TeraPool, the kernels of MMSE are implemented for a single Snitch core, and an independent MMSE problem is then distributed in parallel to each core. The single-core MMSE is implemented in different arithmetic precisions, as this is a key exploration parameter for trading off functional accuracy with execution speed and energy.

1) *16bHalf*: uses the RISC-V *zhinx* set. We load 16b words. A complex MAC requires four `fmadd.h` instructions.

2) *16bwDotp*: uses the *wDotp* extension of the *Smallfloat* set, with 32b accumulation. We load 32b words, and we use two *wDotps* and a *SIMD shuffle* instruction from *Xpulpimg* set to implement the complex MAC, as described in Figure 3.

3) *16bCDotp*: uses the complex dotproduct operation. The multiplication internal precision is 32b, and real-imaginary accumulators are in 16b, fitting a 32b word.

4) *8bQuarter*: uses *Smallfloat* 1b sign, 4b exponent, 2b mantissa FP format. After the execution of the $\hat{H}^H \hat{H}$ matrix product and of the $\hat{H}^H y$ MVM, we cast the outputs to 16b to solve the $G\hat{x} = z$ linear system in higher numerical precision.

5) *8bwDotp*: uses *Smallfloat* *wDotp* for the matrix products and the MVM, with accumulators in 16b. A complex MAC re-

quires a *wDotp* instruction and a *SIMD shuffle* instruction from the *Xpulpimg* set, as described in Figure 3. The linear system is solved in 16b precision.

For the parallel execution, each core runs an independent MMSE problem (1024 in total). The operands are allocated in the shared memory of TeraPool to reduce the cores' contentions to shared interconnect resources [25]. In Figure 4, we represent TeraPool cores and the corresponding input-outputs of independent MIMO problems. The elements of vectors in Equation (1) (y , σ^2 , $\hat{x}$, and $\hat{H}$, flattened to a one-dimension array in row-major order) lay in consecutive addresses of L1. This matches the allocation of data in L2 and does not require relocating elements after explicit DMA transfers to L1. On the contrary, intermediate outputs of the computation L matrix and G matrix have rows on different memory rows. This keeps the data local to the processing cores. For example, in the 4x4 MIMO, each core sends requests to different banks, fetching y , σ^2 , $\hat{x}$, and G , and in the 32x32 MIMO only 8 cores contend for the same scratchpad banks.

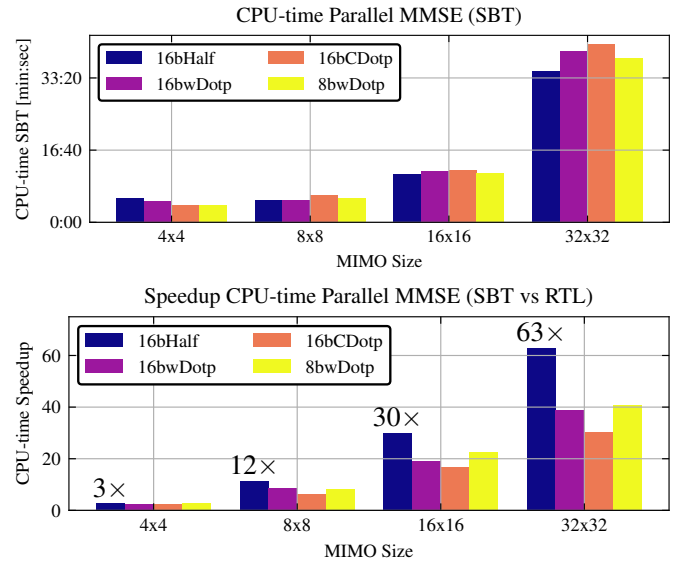


Fig. 5. Parallel MMSE in different precisions and input sizes: speedup on 128-cores AMD EPYC-7742 of multi-thread Banshee simulation against single-thread RTL simulation with QuestaSim-2022.3.

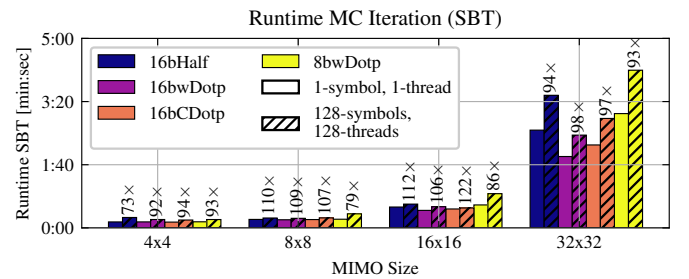


Fig. 6. Batched execution on a Snitch of $N_{SC} = 1638$ MMSE problems of an OFDM-symbol: single-thread runtime of Banshee simulation and multi-thread simulation of independent symbols with corresponding speedup, on a 128-cores AMD EPYC-7742 server.

V. BER & RUNTIME ANALYSIS

This section explains how SBT can be used for fast design-space exploration of MMSE in different arithmetic precisions. It presents Banshee runtime, then shows how to measure the KPIs of software-defined MIMO applications.

A. Simulator runtime performance

We execute 1024 MMSE problems in parallel on 1024 TeraPool cores. We simulate a total of 1024 independent instruction streams. The experiment runs on a 128-CPU server processor at 3.25GHz (AMD EPYC-7742). Each CPU has 32MiB L3-cache and shares 1TB of system memory. We compare the total CPU-time of a multi-thread Banshee simulation with the CPU-time of a single-thread RTL simulation (Questasim-2022.3 being intrinsically single-threaded). For multi-thread, we average the results of 10 independent runs to filter the effect of competing processes on the server. Figure 5 shows the runtime (top) and speedup (bottom) results: the total CPU-time for a Banshee call amounts to average < 35 min, while the average wall-clock runtime is < 2 min:45s, over all the input sizes. Comparing CPU-times, Banshee simulation is up to $63\times$ faster than single-thread QuestaSim-2022.3 RTL simulation. Comparing wall-clock runtimes, Banshee multi-thread simulation is up to $1582\times$ faster than single thread RTL simulation.

To speed up the simulation, the MC simulation of independent MMSE problems is batched on a single TeraPool core. We consider a New Radio (NR) transmission in a 50MHz bandwidth, with $N_{SC} = 1638$, 30kHz subcarrier spacing, and 0.5ms TTI duration. We simulate a MC iteration, consisting of $N_{SC} = 1638$ MMSE problems running on a single Snitch, with a single AMD EPYC-7742 thread. The maximum speed for a single-threaded Banshee simulation of a single Snitch is 3.57 Million Instructions Per Second (MIPS). The runtime in Figure 6, shows < 3 min per MC iteration, and down to 9.44s for an input 4x4 MIMO, compared to 13h:44min required by RTL simulation of the same binary ($5237\times$ faster).

Independent OFDM symbols can be easily parallelized over the 128 cores of the server. Figure 6 shows the runtime with 128 active threads and the speedup compared to a single thread. We measure 12.5s-4min:10s simulation runtime per 128 MC iterations, depending on the MIMO size and arithmetic precision: 73 - $121\times$ speedup compared to single-thread execution. The achieved simulation throughput is fully compatible with a fast HW-in-the-loop exploration of the E2E 5G application under exam.

B. MMSE cycle-count on TeraPool

Figure 7 reports the cycle-count of the complete parallel *16bHalf* MMSE application, measured by RTL simulation in QuestaSim-2022.3. The cycle-accurate RTL simulation gives precise information on the speedup of lower arithmetic precisions compared to *16bHalf*. SIMDs reduce the total number of instructions issued. In particular, for a 32x32 input, the ratio between instructions issues in the *16bHalf* compared to *16bwDotp*, *16bCDotp*, and *8bwDotp* corresponds to $1.15\times$,

$1.84\times$, and $1.37\times$. These values are slightly smaller than the measured speedup ($1.29\times$, $1.86\times$, $1.38\times$): requiring fewer loads, compared to *16bHalf*, SIMD instructions also reduce the stalls from contentions in the shared interconnect.

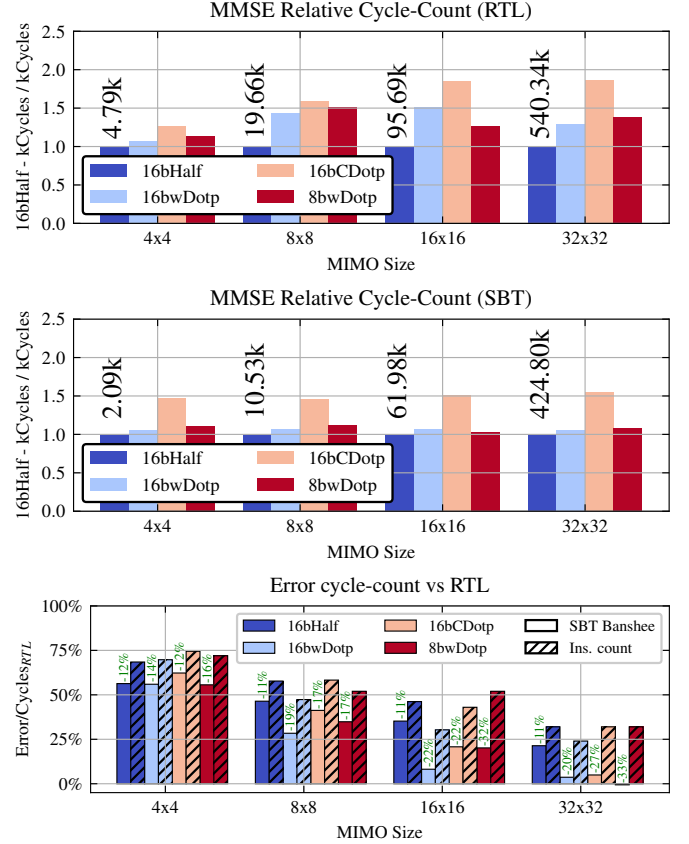


Fig. 7. Parallel implementation of the MMSE in different precisions for various sizes of MIMO. Error between the cycles measured by RTL simulation with QuestaSim-2022.3 and estimated by Banshee or by counting instructions.

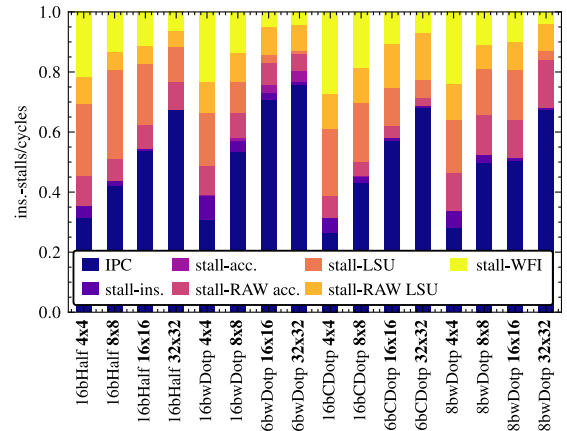


Fig. 8. Breakdown of instructions and architectural stalls over the cycle-count. Result of cycle-accurate experiments in QuestaSim-2022.3.

The breakdown of instructions and stalls over the cycle count for different arithmetic precisions and input problem sizes is in Figure 8. We measure few *stall-ins.* for I\$ refill and

stall-acc., occurring when the pipelines of Snitch functional units are full. Loops are unrolled to minimize RAW stalls, with increasing benefits at higher problem sizes. The amount of stalls due to contentions in the interconnect (*stall-LSU*) is the highest for the low arithmetic intensity *16bHalf* application. Finally, *stall-WFI* refers to the time spent by cores idling at the synchronization during the fork-join MMSE application.

Figure 7 reports a comparison between the cycle count measured by RTL simulation, the cycle count estimated by Banshee, and a rough estimation based on the instruction count. Privileging multi-thread fast simulation of parallel programs on the host, Banshee omits priorities between loads&stores targeting the same memory bank or Tiles’ shared-ports to interconnects, as well as priorities between atomic memory operations, used to implement synchronization barriers. Modeling these effects would require expensive synchronization between hardware threads or computationally heavy statistical models. Our approximate solution, oriented to simulation speed, is conservative in assigning statically to all the transactions the largest memory access latency without contentions (9 cycles).

On average, we obtain 30% error compared to RTL simulation cycle-count. The largest errors measured (20-62%) correspond to the *16bHalf* implementation, which loads&stores separately the real-imaginary parts of the operands (twice the number of memory operations, compared to other kernels). Small MIMO problems with large synchronization overhead are more affected by the permissive latency modeling of atomics. Nevertheless, in this unfavorable case, Banshee obtains 12-16% improvement (in green) on the rough estimation considering the instruction count only. For a large 32x32 MIMO input, the error of Banshee is only 0.6-20%.

Despite the cycle performance over-estimation, in Figure 7 we note that measuring the cycle count in Banshee does not invalidate the key insight on the speedup of SIMD implementations, compared to *16bHalf*. The values measured for 32x32 inputs, 1.05 $\times$, 1.54 $\times$, 1.07 $\times$, confirm *16bCDotp* as the fastest option, followed by *8bwDotp* and *16bwDotp*. The tool is useful for rough estimations of the E2E cycle count. Targeting an open-source DUT facilitates further calibration and reduction of performance over-estimation based on RTL simulations results.

C. BER extraction via fast MC simulation

A classic telecommunications KPI is the Bit Error Rate (BER), which measures the ratio of bit errors compared to the original transmitted bit sequence. We compute the BER of the MMSE implemented on TeraPool simulating MC iterations with Banshee and leveraging its bit-true functional modeling accuracy. For different input Signal to Noise Ratio (SNR), we iterate to a target error count. We explored the performance of the MMSE detector in different arithmetic precisions according to three variables: MIMO problem size, bit-sequence modulation, and wireless channel type.

The input bit-sequence maps to different QAM constellations. In Figure 9, we compare the BER vs. SNR, obtained for a 4x4 or a 32x32 MMSE problem with 16QAM and

a 64QAM modulation. We observe the *16bHalf*, *16bwDotp*, and *16bCDotp* implementations achieving the same results as the *64bDouble* Python model. The *8b* implementations both suffer the truncation of results precision before the 16b matrix inversion, corresponding to a 10 $\times$ loss at 18dB.

In Figure 10, we examine the BER of the receiver, assuming a flat-fading Rayleigh channel. While the AWGN channel assumes zero attenuation and interference from other transmitters, the Rayleigh channel considers the effects of multi-path fading and models a 5G-MIMO transmission more closely. The MC co-simulation of TeraPool and the wireless channel gives immediate feedback on the performance of low-bit arithmetic precision RISC-V extensions on the full wireless application: only the *16bwDotp* and *16bCDotp* implementations follow the performance of the *64bDouble* golden model. The fast co-simulation approach revealed the benefits of accumulating in 32b (*16bwDotp*), or execution of the complex MAC in 32b internal precision (*16bCDotp*).

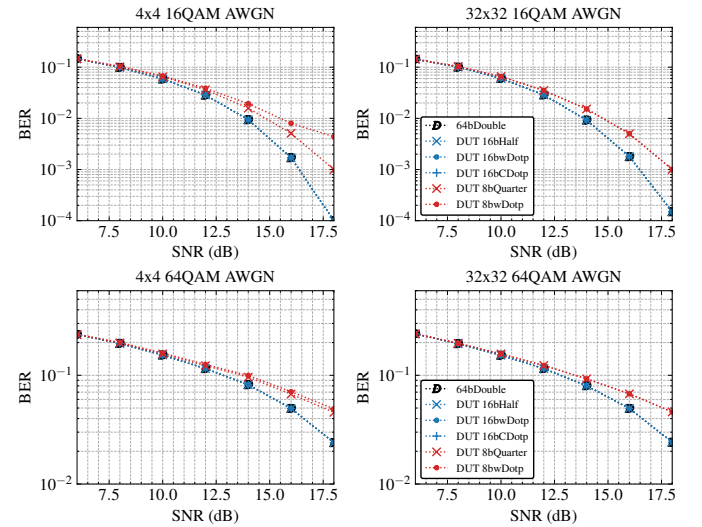


Fig. 9. BER for 16QAM and 64QAM modulation, AWGN channel.

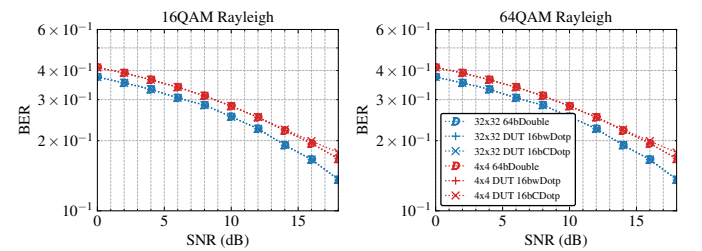


Fig. 10. BER for 16QAM and 64QAM modulation, Rayleigh channel.

VI. CONCLUSIONS

This work presented an open-source SBT-based lightweight simulator coupled to wireless channel models to extract the E2E performance of RISC-V SDR-transceivers. The simulation framework fulfills the requisites [7] to model BS programmable receivers in large-scale RAN testbed. SBT offers

fast, functional modeling of SDR workloads on the open-source TeraPool 1024-core cluster, accounting for the low-bit precision of the hardware. It enables fast design space exploration of E2E 5G transmissions, considering different modulation schemes and wireless channels: MMSE on a 5G OFDM-symbol is simulated in only 9.5s-3min, depending on the input MIMO size, three orders of magnitude faster than RTL simulation. Independent symbols are parallelized on up to 128 threads of a COTS server with $73\text{-}121\times$ speedup compared to single-thread simulation. The translation approach is agnostic of the algorithm used, which makes the method flexible to an evolving telecommunication standard. Finally, augmenting the simulator with a simple hardware timing model gives first-order estimates of the runtime on the DUT, with only 30% average error, for different MIMO sizes. The fully open-sourced RTL of the DUT ease further calibration and reduction of the performance over-estimation.

ACKNOWLEDGMENT

This work is funded in part by the COREnext project supported by the EU Horizon Europe research and innovation program under grant agreement No. 101092598.

REFERENCES

- [1] P. Jonsson and et al., "Ericsson Mobility Report 2024," Ericsson, Tech. Rep., 2024. [Online]. Available: <https://www.ericsson.com/en/reports-and-papers/mobility-report>
- [2] A. Kamaleldin, S. Hesham, and D. Göhringer, "Towards a Modular RISC-V Based Many-Core Architecture for FPGA Accelerators," *IEEE Access*, vol. 8, pp. 148 812–148 826, 2020.
- [3] Y. Zhang, S. Riedel, M. Bertuletti, A. Vanelli-Coralli, and L. Benini, "TeraPool-SDR: An 1.89TOPS 1024 RV-Cores 4MiB Shared-L1 Cluster for Next-Generation Open-Source Software-Defined Radios," in *proceeding of 2024 Great Lakes Symposium on VLSI (GLSVLSI)*, 2024, p. 86–91.
- [4] S. Rajagopal, "EDGE Q 5G with an EDGE," in *2021 IEEE Hot Chips 33 Symposium (HCS)*, 2021, pp. 1–13.
- [5] 3GPP, "NR and NG-RAN Overall Description," 3rd Generation Partnership Project (3GPP), Technical Specification (TS) 38.211, 2017, Release 17.
- [6] L.-H. Shen, K.-T. Feng, and L. Hanzo, "Five Facets of 6G: Research Challenges and Opportunities," *ACM Comput. Surv.*, vol. 55, no. 11, pp. 1–39, 2023.
- [7] R. Wittig, A. Goens, C. Menard, E. Matus, G. P. Fettweis, and J. Castrillon, "Modem Design in the Era of 5G and Beyond: The Need for a Formal Approach," in *2020 27th International Conference on Telecommunications (ICT)*, 2020, pp. 1–5.
- [8] H. Rezgui, F. Rouissi, and A. Ghazel, "Design Methodology Proposal of Digital Predistorter Using Matlab and Modelsim Cosimulation," in *2018 6th International Conference on Multimedia Computing and Systems (ICMCS)*, 2018, pp. 1–5.
- [9] C. Y. Park and J. Kim, "Event-driven modeling and simulation of 5g nr-band rf transceiver in systemverilog," in *SMACD / PRIME 2021; International Conference on SMACD and 16th Conference on PRIME*, 2021, pp. 1–4.
- [10] A. Barreateau, S. Le Nours, and O. Pasquier, "A Case Study of Simulation and Performance Evaluation of a SDR Baseband Architecture," *Journal of Signal Processing Systems*, vol. 73, no. 3, pp. 267–279, 2013.
- [11] S. Cheng, O. A. Topal, M. Ozger, C. Cavdar, O. Edfors, and L. Liu, "A Hardware-in-the-Loop Simulator for mmWave Massive MIMO Using PYNQ Framework," in *2023 IEEE 66th International Midwest Symposium on Circuits and Systems (MWSCAS)*, Aug. 2023, pp. 698–702, iSSN: 1558-3899. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10405907>
- [12] "QuestaSim." [Online]. Available: <https://eda.sw.siemens.com/en-US/ic/questa/simulation/advanced-simulator/>
- [13] "Verilator." [Online]. Available: <https://www.veripool.org/verilator/>
- [14] "5G SoC Design: Enabling Virtual Verification with Keysight." [Online]. Available: <https://www.synopsys.com/blogs/chip-design/5g-soc-design-virtual-verification-with-keysight.html>
- [15] G. Miorandi, F. Stefanni, F. Fraccaroli, and D. Quaglia, "A SystemC-based Simulator for design space exploration of smart wireless systems," in *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2018, pp. 666–671.
- [16] B. Beichler, M. Rethfeldt, H. Raddatz, L. Steffen, D. Timmermann, and C. Haubelt, "A SystemC Model of the IEEE 802.11 EDCA Protocol for Virtual Prototyping," in *2021 International Symposium on Networks, Computers and Communications (ISNCC)*, 2021, pp. 1–7.
- [17] G. Gailliard, E. Nicolle, M. Sarlotte, and F. Verdier, "Transaction Level Modelling of SCA Compliant Software Defined Radio Waveforms and Platforms PIM/PSM," in *2007 Design, Automation & Test in Europe Conference & Exhibition*, 2007, pp. 1–6.
- [18] A. Lonardi and G. Pravadelli, "On the Co-simulation of SystemC with QEMU and OVP Virtual Platforms," in *VLSI-SoC: Internet of Things Foundations*, L. Claesen, M.-T. Sanz-Pascual, R. Reis, and A. Sarmiento-Reyes, Eds. Cham: Springer International Publishing, 2015, pp. 110–128.
- [19] S. Riedel, F. Schuiki, P. Scheffler, F. Zaruba, and L. Benini, "Banshee: A Fast LLVM-Based RISC-V Binary Translator," in *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 2021, pp. 1–9.
- [20] J. Hoydis, S. Cammerer, F. A. Aoudia, A. Vem, N. Binder, G. Marcus, and A. Keller, "Sionna: An Open-Source Library for Next-Generation Physical Layer Research," 2023. [Online]. Available: <https://arxiv.org/abs/2203.11854>
- [21] "Announcing public review for RISC-V standard extensions Zfinx, Zdinx, Zhinx, and Zhinxmin." [Online]. Available: <https://riscv.org/blog/2021/08/announcing-public-review-for-risc-v-standard-extensions-zfinx-zdinx-zhinx-and-zhinxmin/>
- [22] G. Tagliavini, S. Mach, D. Rossi, A. Marongiu, and L. Benini, "Design and Evaluation of SmallFloat SIMD extensions to the RISC-V ISA," in *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2019, pp. 654–657.
- [23] L. Bertaccini, G. Paulin, M. Cavalcante, T. Fischer, S. Mach, and L. Benini, "MiniFloats on RISC-V Cores: ISA Extensions with Mixed-Precision Short Dot Products," *IEEE Transactions on Emerging Topics in Computing*, pp. 1–16, 2024.
- [24] S. Shahabuddin, M. H. Islam, M. S. Shahabuddin, M. A. Albreem, and M. Juntti, "Matrix Decomposition for Massive MIMO Detection," in *2020 IEEE Nordic Circuits and Systems Conference (NorCAS)*, 2020, pp. 1–6.
- [25] M. Bertuletti, Y. Zhang, A. Vanelli-Coralli, and L. Benini, "Efficient Parallelization of 5G-PUSCH on a Scalable RISC-V Many-Core Processor," in *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2023, pp. 1–6.